2D / 3D diffusion problem with point source

This notebook is based on the work of Lorena Barba, see https://github.com/Angelo1211/CFDPython. and modified by Paul Van Liedekerke (2025).

In this notebook, we'll look at the finite differences solution approach for the 2D/3D diffusion problem with boundary conditions.

As example, we consider of a 3D box filled with cells that consume some molecule. The boundary concentration of the box is fixed.

We'll solve the **time dependent** problem with an explicit solver (to be replaced by a Crank-Nicholson solver !) and the **steady state** problem (Poisson equation). Both solutions are compared for consintency.

2D Diffusion

First, let's look the 2D diffusion problem with a point source and DC and NM boundary conditions.

Tthe 2D-diffusion equation for a scalar field with a point source P at x_0, y_0 :

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial x^2} + \nu \frac{\partial^2 u}{\partial y^2} + P\delta(x - x_0, y - y_0)$$

▶ `Click here to see the solution`

In [194…
```
import numpy as np
from matplotlib import pyplot as pt
from matplotlib import cm
%matplotlib inline
```

In [195…
```
#simulation settings
x_domain=2
y_domain=2
t_domain=10

nx = 81
ny = 81

nu=0.05
sigma = 0.9
```

In [196…
```
#initial conditions
dx = x_domain / (nx - 1)
dy = y_domain / (ny - 1)

u_0= np.zeros((ny, nx))
u_0[int(.5 / dy):int(1 / dy + 1),int(.5 / dx):int(1 / dx + 1)] = .0

Pposx =40
Pposy =40
```

```
P_0 = np.zeros((ny, nx))
P_0[Pposx,Pposy] = .0
```

In [197…
```
def constant_BC(u_old, location, constant=1):
    '''
    Function to implement the BCs.
    A BC is required on the bottom, top, left and right.
    We assume these are the values that location can take on.
    This function might look like an overkill,
    however, for more complex problems, you will be happy you have
    implemented the boundary conditions in this manner.
    '''

    if location == 'bottom':
        return constant

    elif location == 'top':
        return constant

    elif location == 'left':
        return constant

    elif location == 'right':
        return constant

    else:
        pass


    return
```

In [198…
```
def diffusion_2D(nx, x_domain, ny, y_domain, t_domain, sigma, nu, BCs, u_0, P_0):

    dx = x_domain / (nx - 1)
    dy = y_domain / (ny - 1)

    dt=sigma / (1/dx**2 + 1/dy**2) / nu / 2
    print (dt)
    nt=int(t_domain/dt)

    u_new = u_0.copy()

    for n in range(nt):
        u_old = u_new.copy()
        u_new[1:-1, 1:-1] = (u_new[1:-1,1:-1]
                             + nu * dt / dx**2 * (u_new[1:-1, 2:]
                                                  - 2 * u_new[1:-1, 1:-1]
                                                  + u_new[1:-1, 0:-2])
                             + nu * dt / dy**2 * (u_new[2:,1: -1]
                                                  - 2 * u_new[1:-1, 1:-1]
                                                  + u_new[0:-2, 1:-1])) - P_0[1:-1,1:-1]

        u_new[0, :] = BCs(u_old, 'bottom')
        u_new[-1, :] = BCs(u_old, 'top')
        u_new[:, 0] = BCs(u_old, 'left')
        u_new[:, -1] = BCs(u_old, 'right')

    return u_new, dt, nt
```

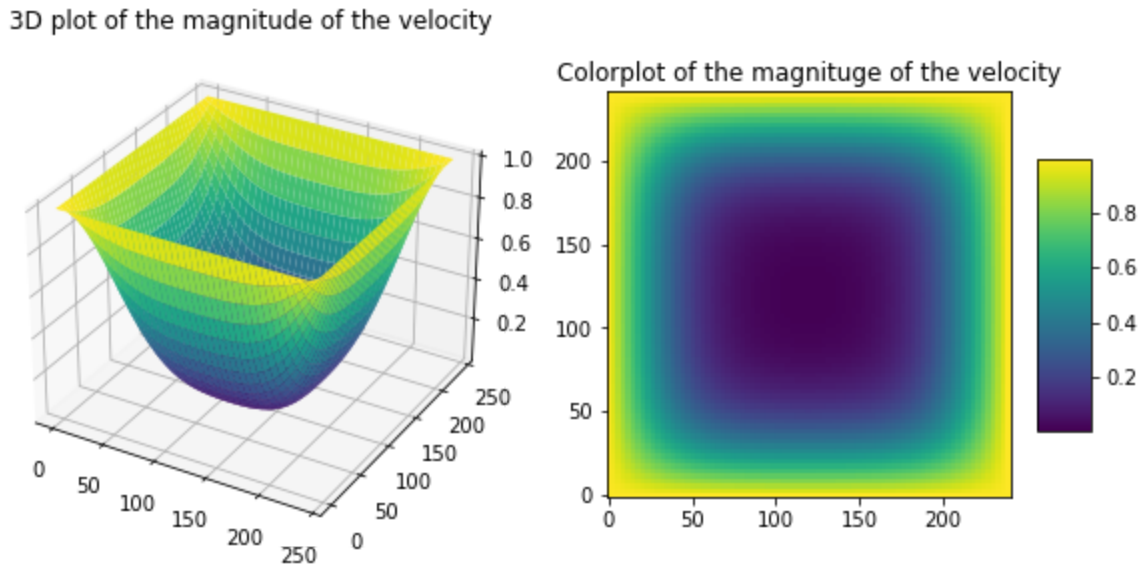
In [199…
```
#If you did well, this piece of code should run smoothly
t_domain=1
u_diffusion_2D, dt, nt=diffusion_2D(nx, x_domain, ny, y_domain, t_domain, sigma, nu, con
```

```
0.0028125000000000003
```

In [200…
```
fig=pt.figure(figsize=(10, 5))

ax1=fig.add_subplot(1, 2, 1, projection="3d")
X, Y = np.meshgrid(x, y)
b=ax1.plot_surface(X, Y, u_diffusion_2D, cmap=cm.viridis)
ax1.set_title('3D plot of the magnitude of the velocity')

ax2=fig.add_subplot(1, 2, 2)
ax2.pcolor(X, Y, u_diffusion_2D)
ax2.set_title('Colorplot of the magnituge of the velocity')
ax2.set_aspect('equal', 'box')
fig.colorbar(b, shrink=0.5, aspect=5)
```

```
C:\Users\pvliedek\AppData\Local\Temp\ipykernel_72632\139282867.py:12: MatplotlibDeprecat
ionWarning: Starting from Matplotlib 3.6, colorbar() will steal space from the mappabl
e's axes, rather than from the current axes, to place the colorbar.  To silence this war
ning, explicitly pass the 'ax' argument to colorbar().
  fig.colorbar(b, shrink=0.5, aspect=5)
```

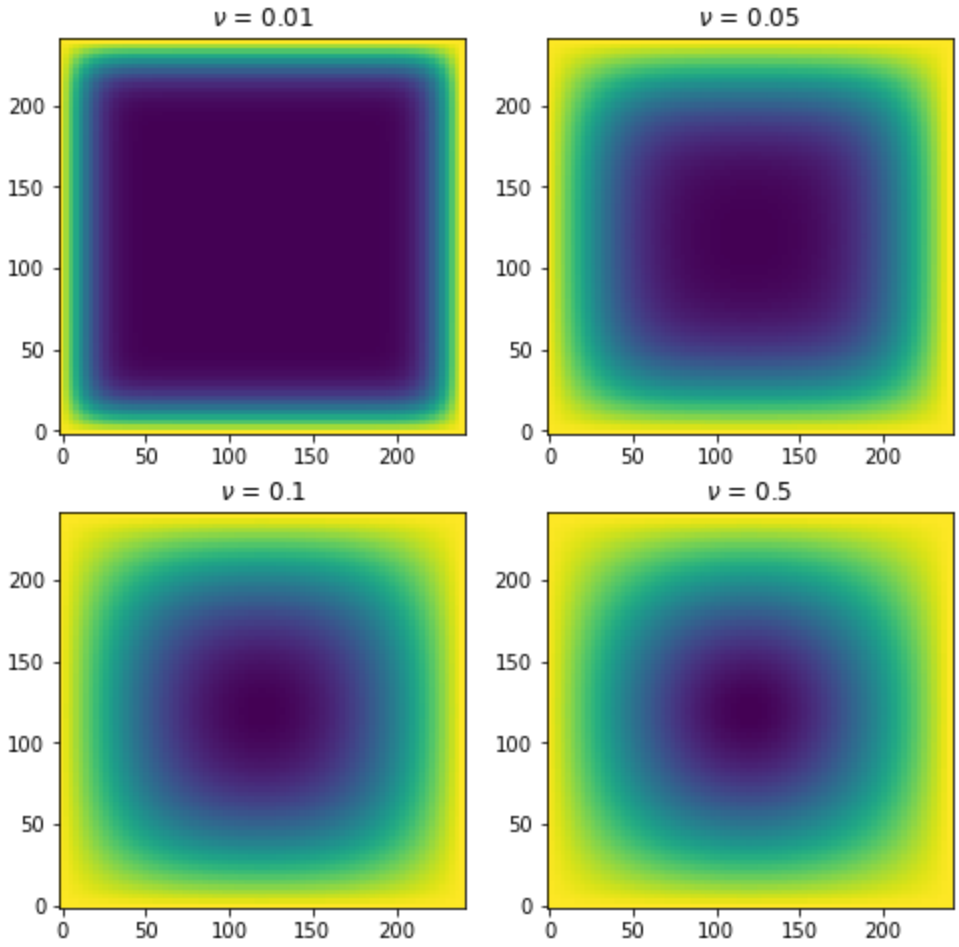
Out[200]:
```
<matplotlib.colorbar.Colorbar at 0x1f0f0688970>
```



EXERCISE: Play around with the diffusion coefficient

In [201…
```
fig, ax= pt.subplots(2,2, figsize=(8,8))

list_of_nus=[0.01,0.05, 0.1, 0.5]
#Choose 4 different values for the diffusion in the list above
#The code below will nicely plot them for you
for i,nu in enumerate(list_of_nus):
    u_diffusion_2D,dt, nt=diffusion_2D(nx, x_domain, ny, y_domain, t_domain, sigma, nu,
    ax[i//2,i%2].pcolor(X, Y, u_diffusion_2D)
    ax[i//2,i%2].set_title(r'$\nu$ = {}'.format(nu))
```

```
0.014062500000000002
0.0028125000000000003
0.0014062500000000002
0.00028125000000000003
```



In this case we assume an isotropic diffusion coefficiënt (diffusion is the same in both x-y direction). In some cases (porous media) this is not the case.

In [ ]:

3D Diffusion in a box filled with cells

We now solve the 3D diffusion problem in a box with point sources and Dirichlet boundary conditions.

The 3D-diffusion equation for a scalar field with a point source P at x_0, y_0 :

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial x^2} + \nu \frac{\partial^2 u}{\partial y^2} + \nu \frac{\partial^2 u}{\partial z^2} + P\delta(x - x_0, y - y_0, z - z_0)$$

▶ `Click here to see the solution`

We assume a box:

- Domain: 240 x240x240 um
- BC: 10 uM on all sides
- Source $P_0 = 2000uM/min$ for all cells
- Diffusion coefficient : 2000 um^2/s

In [186…

```
#simulation settings
x_domain=240
```

```
y_domain=240
z_domain=240
t_domain=10

nx = 81
ny = 81
nz = 81

nu=2000
sigma = 0.9
```

In [187…

```
#initial conditions
dx = x_domain / (nx - 1)
dy = y_domain / (ny - 1)
dz = z_domain / (nz - 1)

u_0= 10*np.ones((ny, nx,nz))
#u_0[int(.5 / dy):int(1 / dy + 1),int(.5 / dx):int(1 / dx + 1), : ] = .0

Pposx =40
Pposy =40
Pposz =40

P_0 = np.zeros((ny, nx,nz))


for i in range(1000):
 indices = np.random.randint(2,nx-2, size=(3))
 P_0[indices[0],indices[1],indices[2]] += 2000


print(sum(sum(sum(P_0))))     # checking total consumption
```

```
2000000.0
```

In [188…

```
def constant_BC_3D(u_old, location, constant=10):
    '''
    Function to implement the BCs.
    A BC is required on the bottom, top, left and right.
    We assume these are the values that location can take on.
    This function might look like an overkill,
    however, for more complex problems, you will be happy you have
    implemented the boundary conditions in this manner.
    '''

    if location == 'bottom':      # Z
        return constant

    elif location == 'top':
        return constant

    elif location == 'left':     # Z
        return constant

    elif location == 'right':
        return constant

    elif location == 'front':        # Z
        return constant

    elif location == 'back':
        return constant

    else:
        pass
```

```
    return
```

In [189…
```
def diffusion_3D(nx, x_domain, ny, y_domain, nz, z_domain, t_domain, sigma, nu, BCs, u_0

    dx = x_domain / (nx - 1)
    dy = y_domain / (ny - 1)
    dz = z_domain / (nz - 1)

    dt=0.5* sigma / ((1/dx**2 + 1/dy**2 + 1/dz**2)) / nu
    print (dt)
    nt=int(t_domain/dt)

    u_new = u_0.copy()

    for n in range(nt):
        u_old = u_new.copy()

        u_new[1:-1, 1:-1, 1:-1] = (u_new[1:-1,1:-1, 1:-1]  + nu * dt / dx**2 * (u_new[1:
                                                            + u_new[1:-1, 0:-2,1:-1]) + nu
                                                            + u_new[0:-2, 1:-1,1:-1]) + nu



        u_new[0, :,:] = BCs(u_old, 'bottom')
        u_new[-1, :,:] = BCs(u_old, 'top')
        u_new[:, 0,:] = BCs(u_old, 'left')
        u_new[:, -1,:] = BCs(u_old, 'right')
        u_new[:, :,0] = BCs(u_old, 'front')
        u_new[:,:,-1] = BCs(u_old, 'back')

    return u_new, dt, nt
```

In [190…
```
#If you did well, this piece of code should run smoothly
# this may run for  a while...

t_domain=30  # end time   7.74166
u_diffusion_3D, dt, nt=diffusion_3D(nx, x_domain, ny, y_domain,nz, z_domain, t_domain, s
```

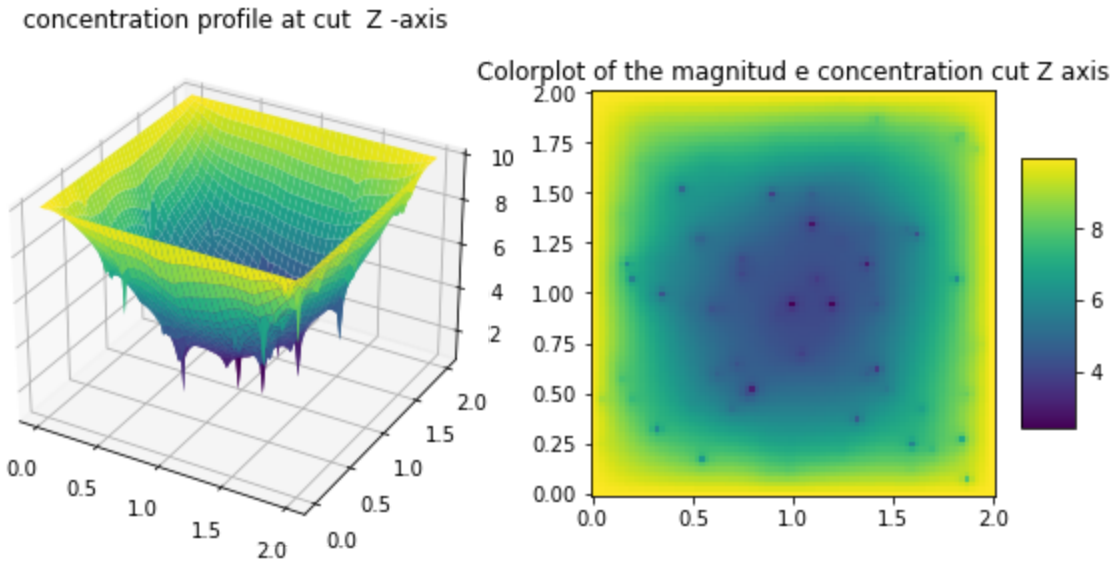
```
0.000675
```

In [202…
```
print("timestep:", dt)

fig=pt.figure(figsize=(10, 5))
u_cut=u_diffusion_3D[:,:,40]
mean_u = np.mean(u_diffusion_3D)

print("mean box concentration: ", mean_u)

x=np.linspace(0,x_domain,nx)
y=np.linspace(0,y_domain,ny)
ax1=fig.add_subplot(1, 2, 1, projection="3d")
X, Y = np.meshgrid(x, y)
b=ax1.plot_surface(X, Y, u_cut, cmap=cm.viridis)
ax1.set_title('concentration profile at cut  Z -axis')

ax2=fig.add_subplot(1, 2, 2)
ax2.pcolor(X, Y, u_cut)
ax2.set_title('Colorplot of the magnitud e concentration cut Z axis')
ax2.set_aspect('equal', 'box')
fig.colorbar(b, shrink=0.5, aspect=5)
```

```
timestep: 0.00028125000000000003
mean box concentration:  7.536333174769088
C:\Users\pvliedek\AppData\Local\Temp\ipykernel_72632\2680570816.py:20: MatplotlibDepreca
```

```
tionWarning: Starting from Matplotlib 3.6, colorbar() will steal space from the mappabl
e's axes, rather than from the current axes, to place the colorbar.  To silence this war
ning, explicitly pass the 'ax' argument to colorbar().
  fig.colorbar(b, shrink=0.5, aspect=5)
```

Out[202]:
```
<matplotlib.colorbar.Colorbar at 0x1f0f48dcfd0>
```



3D Diffusion Backward Differencing

In [348…
```
# todo..
```

3D Diffusion Crank-Nicholson

In [350…
```
# todo..
```

In [ ]:
```

```

3D Diffusion Steady state solution

The discretized equation is:

$$\frac{p^n_{i+1,j,k} - 2p^n_{i,j,k} + p^n_{i-1,j,k}}{\Delta x^2} + \frac{p^n_{i,j+1,k} - 2p^n_{i,j,k} + p^n_{i,j-1,k}}{\Delta y^2} + \frac{p^n_{i,j,k+1} - 2p^n_{i,j,k} + p^n_{i,j,k-1}}{\Delta z^2} = P_{i,j,k}$$

Notice that the Laplace Equation does not have a time dependence — there is no p^{n+1}. Instead of tracking a wave through time (like in the previous steps), the Laplace equation calculates the equilibrium state of a system under the supplied boundary conditions.

If you have taken coursework in heat transfer, you will recognize the Laplace equation just as a steady-state heat equation.

Instead of calculating where the system will be at some time t, we will iteratively solve for $p^n_{i,j}$ until it meets a condition that we specify. The Laplace equation is a **boundary-value problem** i.e. a differential equation

subjected to constraints called boundary conditions. The system will reach equilibrium only as the number of iterations tends to ∞, but we can approximate the equilibrium state by iterating until the change between one iteration and the next is *very* small.

Let's rearrange the discretized equation, solve for $p^n_{i,j}$, and call it $p^{n+1}_{i,j}$:

$$p^{n+1}_{i,j} = \frac{\Delta y^2\Delta z^2(p^n_{i+1,j} + p^n_{i-1,j}) + \Delta x^2\Delta z^2(p^n_{i,j+1} + p^n_{i,j-1}) + \Delta x^2\Delta y^2(p^n_{i,j,k+1} + p^n_{i,j,k-1}) - P_{i,j,k}\Delta x^2\Delta y^2\Delta z^2}{2(\Delta y^2\Delta z^2 + \Delta x^2\Delta z^2 + \Delta x^2\Delta y^2)}$$

In [174…
```
import numpy as np
from matplotlib import pyplot as pt
from matplotlib import cm
%matplotlib inline
```

In [175…
```
def laplace_3D(dx, dy,dz, n_iter,nu, u_0,P_0, BCs):
    '''
    Function to solve Laplace's equation in 3D,
    given the spatial discretisation, the number of
    iterations, the initial condition and the
    boundary conditions
    '''
    u_new=u_0.copy()

    for n in range(n_iter):
        u_old = u_new.copy()
        u_new[1:-1, 1:-1,1:-1] =((dy**2 * dx**2 * (u_old[1:-1, 2:, 1:-1] + u_old[1:-1, 0
                            +dx**2 * dz**2 * (u_old[2:, 1:-1,1:-1] + u_old[0:-2, 1:-1,1:
                            /(2 * (dy**2 * dz**2 + dx**2 *dz**2 + dx**2 *dy**2)))

        u_new[0, :,:] = BCs(u_old, 'bottom')
        u_new[-1, :,:] = BCs(u_old, 'top')
        u_new[:, 0,:] = BCs(u_old, 'left')
        u_new[:, -1,:] = BCs(u_old, 'right')
        u_new[:, :,0] = BCs(u_old, 'front')
        u_new[:,:,-1] = BCs(u_old, 'back')

    return u_new
```

In [176…
```
def boundary_condition_wrap(y):

    '''
    This wrapper function defines and creates a new function.
    This is necessary, as our piece of code for the update equations requires
    a function as argument
    '''

    def boundary_condition(u_old,location,y=y):
        '''
        Function to implement the BCs.
        A BC is required on the bottom, top, left and right.
        We assume these are the values that location can take on.
        '''

        if location == 'bottom':
            return y

        elif location == 'top':
            return y

        elif location == 'left':
```

```
            return y

        elif location == 'right':
            return y

        elif location == 'front':
            return y

        elif location == 'back':
            return y

        else:
            pass

        return

    return boundary_condition
```

In [177…

```
def converge_laplace_3D(x_domain, nx, y_domain, ny, z_domain, nz,nu, u_0, P_0, BCs, thre
    '''
    Function which calls the update function several times.
    At each function call, the previous solution is compared
    to the current solution.
    When the RMSE value is below the given threshold value,
    the solution is assumed to be converged.
    '''
    dx = x_domain / (nx - 1)
    dy = y_domain / (ny - 1)
    dz = z_domain / (nz - 1)

    y = np.linspace(0, y_domain, ny)
    u_c=10

    u_old=u_0.copy()
    u_new=laplace_3D(dx, dy, dz, 1,nu, u_0,P_0, BCs( u_c))

    norm=np.sqrt(np.mean((u_old-u_new)**2))

    total_iter=1

    while norm>threshold:
        u_old=u_new.copy()
        u_new=laplace_3D(dx, dy, dz, 1,nu, u_old,P_0, BCs( u_c))

        norm=np.sqrt(np.mean((u_old-u_new)**2))
        total_iter=total_iter+1

    return u_new, norm, total_iter
```

In [178…

```
#Simulation settings
x_domain=240
y_domain=240
z_domain=240

nx=81
ny=81
nz=81

x = np.linspace(0, x_domain, nx)
y = np.linspace(0, y_domain, ny)
z = np.linspace(0, z_domain, nz)

nu=2000

P_0 = np.zeros((ny, nx,nz))
```

```
for i in range(1000):
 indices = np.random.randint(2,nx-2, size=(3))
 P_0[indices[0],indices[1],indices[2]] += 2000

print(sum(sum(sum(P_0))))
```

```
2000000.0
```

In [179…
```
#initial condition
u_0 = 10 * np.ones((ny, nx,nz))
```

In [183…
```
u_num_Steady, norm, n_iter = converge_laplace_3D(x_domain, nx, y_domain, ny,z_domain, nz
```

In [184…
```
print('RMS difference between 2 subsequent solutions: {}'.format(norm))
print('Number of iterations: {}'.format(n_iter))
```

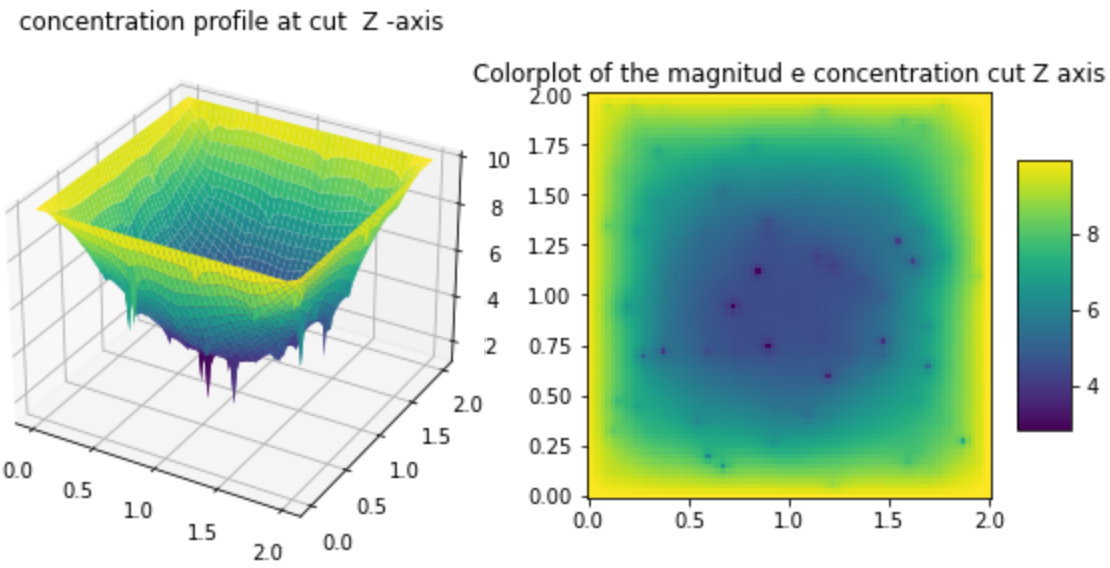
```
RMS difference between 2 subsequent solutions: 9.999988074389736e-06
Number of iterations: 7076
```

In [203…
```
fig=pt.figure(figsize=(10, 5))
u_cut=u_num_Steady[:,:,40]
mean_u = np.mean(u_num_Steady)

print("mean box concentration: ", mean_u)

x=np.linspace(0,x_domain,nx)
y=np.linspace(0,y_domain,ny)
ax1=fig.add_subplot(1, 2, 1, projection="3d")
X, Y = np.meshgrid(x, y)
b=ax1.plot_surface(X, Y, u_cut, cmap=cm.viridis)
ax1.set_title('concentration profile at cut  Z -axis')

ax2=fig.add_subplot(1, 2, 2)
ax2.pcolor(X, Y, u_cut)
ax2.set_title('Colorplot of the magnitud e concentration cut Z axis')
ax2.set_aspect('equal', 'box')
fig.colorbar(b, shrink=0.5, aspect=5)
```

```
mean box concentration:  7.568092288467808
```

```
C:\Users\pvliedek\AppData\Local\Temp\ipykernel_72632\2657222314.py:18: MatplotlibDepreca
tionWarning: Starting from Matplotlib 3.6, colorbar() will steal space from the mappabl
e's axes, rather than from the current axes, to place the colorbar.  To silence this war
ning, explicitly pass the 'ax' argument to colorbar().
  fig.colorbar(b, shrink=0.5, aspect=5)
```

Out[203]:
```
<matplotlib.colorbar.Colorbar at 0x1f0f4ea0fa0>
```



conclusion

The time-dependent solution (after 20 min) and the steady state solution seem to match well (mean concentration in the box $c \approx 7.56$), as should be.

In [ ]: